LOKI BLUEPRINT

A living AI company on your machines - the improved master prompt, the research, the plan, and the tasks for both Zivas. Prepared by Ziva (Windows), 2026-10-14.

1. The improved master prompt

> This is your brief, rewritten as a buildable spec. Give this to any agent (Ziva, Claude Code, LOKI's own) and it knows exactly what to do and what not to.

Mission

Keep improving **LOKI** (github.com/m03dev/loki) into a self-contained AI company the owner runs on a Mac mini and a Windows PC from one code base. It plans work, picks agents and models, runs them, verifies the result, and learns. It is a standalone desktop app - **never** a Godot project.

Hard rules (do not break)

- **No Godot, ever.** The "LOKI World" lives *inside the app* as a 3D WebGL scene. Never create a Godot project; never touch the `ai` project.
- **One version, both machines.** Publish only with `tools/ship.py -m "..."`; it runs every suite, commits, rebases, pushes `master`, and tells the other machine. Never force-push. Tests must pass.
- **Both platforms, always.** No code for one OS without the other; change `setup.sh` and `setup.bat` together; new deps go in `engine/requirements.txt`.
- Never commit `harness_data/` (keys, memory, logs, models). English only.

What to build / keep making real

1. **Settings that feel like a real product.** A Connections *portal*: one card per provider with a toggle, a live *Test* (a real 1-token call), a model picker, *Make default*, *Disconnect*, and a **QR pop-up** so a phone on the same Wi-Fi hands over a key instantly. Sign-in where supported (OpenRouter OAuth, Hugging Face device flow).
2. **The LOKI World, in the app, in 3D.** Agents as characters that move, work, get blocked and speak *only* from real engine events; faint behind the home screen and full size in its own panel. Viewer, never the brain.
3. **An in-depth Agent Builder.** Role, goal, skills (SKILL.md), tools, harness (LOKI / Claude Code / Codex / Hermes / OpenCode), model + fallback, home machine, memory, time limit. "Draft it" builds one from a sentence.
4. **Any model can use any tool.** A prompted tool harness (Hermes `<tool_call>`) so models without native tool calling - MLX, plain llama.cpp, keyless endpoints - get the tools as text and their calls come back as real tool calls.
5. **One-click "Update everything."** Update every linked machine and restart each, then this one; a version that fails to start rolls back and is skipped.
6. **More free model hosts** (section 3) plus local LM Studio, llama.cpp, MLX, vLLM, Jan, Kyojin.
7. **Specialists:** media (yt-dlp), image (Z-Image Turbo / Pollinations), reach (Agent Reach), OpenCode.
8. **Assign work to the two Zivas** (Mac, Windows) in `ZIVA_TASKS.md`.

Excluded on purpose

OBLITERATUS (removes safety refusals from model weights) and Kyojin's "uncensored" presets: **not integrated** (section 3).

2. Where LOKI already is (checked against the code)

Your brief is nearly all *already built* in the repo - the other agent (Claude) landed it. The honest state:

Your ask	Status	Where
One-click update across both machines, with rollback	Built, in CI	`update_all.py` , `updater.py` ; two-machine run = **W7**
Connections portal with toggles/Test/model picker	Built, in CI	`providers_catalog.py` (43 providers), `portal_api.py` , `settings.js`
QR pop-up to connect accounts faster	Built, in CI	`qr.py` , `signin.py` ; real scan = **W3**
More free LLM hosts	Built (43), gaps remain	groups: local / free / keyless / trial / subscription / paid
LOKI World 3D in the app	Built (WebGL), in CI	`world.js` ; WebView2/WKWebView = **W6/M6**
In-depth agent builder + skills + harness	Built, in CI	`agents.py` , Settings > Agents; live run = **W5**
"Any LLM can use any tool" harness	Built, in CI	`tool_harness.py` (Hermes format)
agent-reach / OpenCode / Kyojin	Wired	manifests + runners; Kyojin needs an AMD Strix Halo
yt-dlp (media)	Added to requirements	`runners/media.py`
Laya MLX (faster local first layer on the Mac)	Researched	no MLX build; ONNX CoreML idea = **M5**
Qwen Image 2.1 Turbo	Not yet	**paid** ($0.016/img, 500-img trial) - see section 3

3. Research findings

Free LLM hosts

Permanent free tiers (key, no card): **OpenRouter** (aggregated :free models, 50-1000 req/day), **Google AI Studio / Gemini** (most generous, no card), **Groq** (fastest; 8k tok/min cap), **Cerebras**, **Mistral & Codestral**, **NVIDIA NIM**, **Cloudflare Workers AI**, **Cohere**, **Hugging Face**, **OpenCode Zen**, **Kilo Gateway**, **Vercel AI Gateway**. Trial credits that run out: SambaNova, Hyperbolic, Fireworks, Nebius, Novita, Together, DeepInfra, Scaleway, Baseten, Modal, AI21, Upstage. Gap to close: add DeepInfra, Chutes, Baseten, Modal, Inference.net, AI21, Upstage, NLP Cloud to the catalog.

The "any LLM can use any tool" harness

This is real and standard: **Hermes function calling** - put the tool schemas in a system prompt inside `<tools>`, and the model answers with a JSON object in `<tool_call>` tags that you parse into a normal tool call. LOKI already implements exactly this in `engine/tool_harness.py`; it is what lets MLX, plain llama.cpp and keyless endpoints call tools.

Image generation (your "Qwen 2.1 turbo images free now?")

Not free. Qwen-Image-2.1-Turbo is about **$0.016 per image** via Alibaba Model Studio / QwenCloud; activating gives a one-time **500-image / 90-day** quota. LOKI's current picks are better for zero cost: local **Z-Image Turbo** (CUDA on the PC, MPS on the Mac) with a **Pollinations** keyless fallback. Recommended: add "Alibaba Model Studio (Qwen image)" as a *trial* image provider, not the default.

agent-reach, OpenCode, Kyojin, Laya-MLX, app-idea

Item	What it is	Plan
Agent Reach	Internet reach for agents: web, YouTube, RSS, GitHub (X/Reddit with cookies). Install from GitHub, not the PyPI name.	Wired as the `reach` specialist; make it W5 installable
OpenCode	Open-source coding agent; any provider, incl. OpenCode Zen free models.	Wired as provider + worker (`opencode run`)
Kyojin	ExLlamaV3 server for 100 GB-class models on an AMD Strix Halo (128 GB).	Wired but unusable here; only standard packs, never "uncensored"
Laya MLX	Laya is a typed decision model, not a text model - no MLX build, mlx-lm cannot run it.	Try **ONNX Runtime CoreML** in `laya_server.mjs`; ship only if faster = **M5**
App-idea	An Agent Builder template ("App Idea Shaper").	Already a template; extend the builder
yt-dlp	Video/audio: details, transcripts, search, downloads.	In requirements; install via `setup_extras.py media` = **W5**

OBLITERATUS - recommendation: do NOT integrate

OBLITERATUS is an *abliteration* toolkit that surgically removes the refusal behaviour from a model's weights (SVD on the refusal subspace, analysis modules, six-stage pipeline). It is well known, and the current LOKI repo already excludes it on purpose. I recommend keeping it out: it is outside LOKI's purpose (routing, teams, tools - not model surgery), it is a safety-sensitive component to wire into an autonomous agent, and it needs a GPU training rig. If you want it, keep it a separate opt-in tool the owner runs by hand - not part of the shipped system.

4. What I changed this session

- **Pulled the other agent's work into `master`** - the 3-commit branch `claude/portal-world-updates` (portal, sign-in/QR, agent builder, 3D world, Update everything, specialists), a clean fast-forward.

- **Cleaned the "still in Godot" story**: rewrote the LOKI World sections in `README.md`, `docs/WORLD.md`, `docs/STATUS.md`, `HANDOFF.md`, `docs/ARCHITECTURE.md`, cancelled the Godot task (W9) in `ZIVA_TASKS.md`, and removed the leftover Godot `addons/`.
- **Made the repo private** (`m03dev/loki`) as you asked.
- **Made the test suites hermetic** so the PC can ship: they were failing only because this machine runs Ollama + llama-server and its `.env` points `OPENROUTER_MODEL` at a paid model. All 11 suites now pass.

5. Plan and task assignments

Both Zivas' full checklists live in `ZIVA_TASKS.md` (read it first every session). Summary:

Machine	First tasks
Ziva (Windows) - PC, RTX 3070 Ti, 24/7	W1 update+test · W2 connections Test on every card · W3 QR phone hand-off · W4 OpenRouter sign-in · W5 extras (media, opencode, reach, image-gen) · W6 world screenshot on WebView2 · W7 update-everything · W8 prompted tools on a local model · W10 start at boot
Ziva (Mac) - Mac mini	M1 checkout + `setup.sh` · **M2** **`gh auth login`** (needed now the repo is private) · M3 Tailscale link · M4 MLX · M5 Laya CoreML · M6 world on WKWebView · M7 images on MPS

> **Do this first now that the repo is private:** the Mac must run `gh auth login` (M2) or it can no longer pull updates. The PC is already authenticated (verified).

6. How to work with ZIVA_TASKS.md (readme)

- Add a request as `- [ ] [mac|windows|either] what - why - who asked (date)` under **Requests**; the machines take the oldest open one first.
- When done: tick it, add one line under **Reports** (date, machine, what happened, exact errors), and ship with `python tools/ship.py -m "ziva(<mac|windows>): <task> <result>"`.
- Need the owner (account, phone, purchase, decision)? Move it to **Needs Mohamed** with the exact question - do not guess.

Generated by Ziva (Windows) from a live read of the repo on 2026-10-14. Everything marked "in CI" is proven by an offline suite; "W#/M#" items still need a real machine.